Basics Day 7 — Session 7 (Hours 25–28)

Python Programming (Basic) • Conditionals, while/for Loops, and Loop Patterns

Session 7 Overview

Topics Covered Today

- Hour 25: Conditionals — if/elif/else and boundary handling
- Hour 26: while loops + sentinel patterns
- Hour 27: for loops + range()
- Hour 28: Loop patterns — counters, accumulators, min/max, averages

Hour 25: Conditionals — if/elif/else and Boundaries

Learning Outcomes

- Write clear if/elif/else statements that handle multiple distinct cases
- Explain why the order of elif branches matters when conditions overlap
- Handle boundary values correctly using `<=`, `<`, `>=`, `>`
- Choose between nested and flat if/elif chains based on readability
- Build a shipping calculator that handles all boundary weights correctly

if/elif/else Structure

The Basic Form

```
if condition_1:
    # runs if condition_1 is True
elif condition_2:
    # runs if condition_1 is False and condition_2 is True
elif condition_3:
    # runs if condition_1 and condition_2 are False, condition_3 is True
else:
    # runs if all conditions above are False
```

Key Rule — First Match Wins

Python evaluates conditions **top to bottom** and executes the first block whose condition is True. Once a match is found, all remaining `elif` and `else` blocks are skipped.

Condition Ordering Matters

The Bug — Overlapping Conditions

```
# WRONG: every weight > 10 is also > 0, so "Heavy" is never reached
if weight > 0:
    print("Standard rate")
elif weight > 10:
    print("Heavy rate")   # never executes!
```

The Fix — Most Specific First

```
# CORRECT: check narrowest range first
if weight <= 5:
    rate = 5
elif weight <= 10:
    rate = 10
else:
    rate = 15
```

Boundary Handling

The Off-by-One Problem

A shipping rule says "0 to 5 pounds: $5". Does exactly 5 pounds cost $5 or $10?

```
# <= 5 means "up to and including 5"
if weight <= 5:
    rate = 5
elif weight <= 10:
    rate = 10
else:
    rate = 15
```

Always Test at Boundaries

Test weights 5, 5.01, 10, and 10.01 — these are the values where bugs hide.

- Weight 5 → `weight <= 5` matches → $5
- Weight 5.01 → `weight <= 10` matches → $10

Nested vs Flat Conditionals

Nested — Grows Deeply

```
if is_member:
    if purchase_amount > 100:
        discount = 0.20
    else:
        discount = 0.10
else:
    discount = 0
```

Flat — Easier to Scan

```
if is_member and purchase_amount > 100:
    discount = 0.20
elif is_member:
    discount = 0.10
else:
    discount = 0
```

Demo — Shipping Calculator (Correct Version)

```python
# Shipping calculator - correct version
# Source: Day7_Hour1_Basics.md, section 7.2

weight = float(input("Enter package weight in pounds: "))

if weight <= 0:
    print("Error: weight must be positive.")
elif weight <= 5:
    cost = 5
    category = "Light"
elif weight <= 10:
    cost = 10
    category = "Medium"
else:
    cost = 15
    category = "Heavy"

if weight > 0:
    print(f"Category: {category}")
    print(f"Shipping cost: ${cost}")
```

Demo — Wrong Ordering Bug

```python
# WRONG: overlapping conditions
# Source: Day7_Hour1_Basics.md, section 7.4

weight = float(input("Enter package weight in pounds: "))

if weight > 0:
    cost = 5
    category = "Light"
elif weight > 5:
    cost = 10
    category = "Medium"   # never reached for any valid weight
elif weight > 10:
    cost = 15
    category = "Heavy"    # never reached
```

Testing with `weight = 12` outputs "Light, $5" — **wrong**.

Fix: Either order from most restrictive to least, or use non-overlapping `<=` boundaries.

Lab — Shipping Calculator

Time: 25 minutes

Task: Build a shipping calculator with tiered weight rules.

Rules:

- Weight must be positive — validate and show error if not
- 0 to 5 pounds → $5 flat (Light)
- More than 5, up to 10 pounds → $10 flat (Medium)
- More than 10 pounds → $15 flat (Heavy)

Optional: Add a fragile surcharge — if package is fragile (yes/no), add $3.

Completion Criteria:

- Correct cost for boundary weights 5, 5.01, 10, 10.01

Common Pitfalls — Hour 25

⚠️ **Overlapping conditions** — first `if` is too broad, later `elif` blocks never execute

⚠️ **Wrong comparison operator** — using `<` instead of `<=` shifts the boundary by one unit

⚠️ **No input validation** — not checking for negative or zero weight before computing cost

⚠️ **Inconsistent boundaries** — "5 to 10 pounds" is ambiguous; clarify whether 5 and 10 are included

Quick Check — Hour 25

Question: Given this code, what grade does a score of 85 receive?

```
if score >= 90:
    grade = 'A'
elif score >= 80:
    grade = 'B'
elif score >= 70:
    grade = 'C'
else:
    grade = 'F'
```

What would happen if you reversed the order of the conditions?

Hour 26: while Loops + Sentinel Patterns

Learning Outcomes

- Write a while loop that repeats code until a condition becomes False
- Explain the difference between a while loop and a for loop
- Use `break` to exit a loop early when a goal is achieved
- Use `continue` to skip the rest of the current iteration
- Implement a sentinel pattern — keep prompting until a special value is entered
- Build a password prompt that allows up to 3 attempts

while Loop Basics

Structure

```
while condition:
    # code to repeat
```

Python checks the condition **before each iteration**. If it is False on the first check, the body never runs.

Simple Trace

```
count = 1

while count <= 3:
    print(f"Iteration {count}")
    count = count + 1

print("Done")
```

The Infinite Loop Danger

What Happens When You Forget to Update

```
count = 1

while count <= 3:
    print(f"Iteration {count}")
    # forgot: count = count + 1
```

`count` never changes, so the condition is always True, and the loop runs forever.

If this happens: press `Ctrl+C` in the terminal to interrupt.

Rule: Every while loop must change something that eventually makes the condition False — usually by updating a variable inside the loop body.

break — Exit a Loop Early

```
count = 1

while count <= 10:
    print(f"Iteration {count}")
    if count == 3:
        print("Breaking out of loop")
        break
    count = count + 1

print("Done")
```

When Python hits `break`, it exits immediately — even though `count <= 10` is still True.

Use `break` when: you have achieved your goal early and do not need to keep looping.

continue — Skip to the Next Iteration

```
count = 0

while count < 5:
    count = count + 1
    if count == 3:
        print(f"Skipping {count}")
        continue
    print(f"Processing {count}")

print("Done")
```

Output: `Processing 1`, `Processing 2`, `Skipping 3`, `Processing 4`, `Processing 5`, `Done`

Use `continue` when: you want to skip this case but keep looping (e.g., skip blank lines, re-prompt on invalid input without exiting).

Sentinel Patterns

What Is a Sentinel?

A **sentinel value** signals "stop" — it is never a valid data value.

```
names = []

while True:
    name = input("Enter a name (or 'done' to finish): ")
    if name == 'done':
        break
    names.append(name)

print(f"You entered {len(names)} names.")
```

Pattern: `while True` + `break` on the sentinel. Safe because `break` guarantees exit.

Common sentinels: `'q'`, `'quit'`, `'done'`, empty string `''`, `-1` for positive-number lists.

Demo — Password Prompt with Retry Limit

```python
# Password prompt with retry limit
# Source: Day7_Hour2_Basics.md, section 7.2

correct_password = "python123"
max_attempts = 3
attempts = 0

while attempts < max_attempts:
    attempts = attempts + 1
    password = input(f"Enter password (attempt {attempts} of {max_attempts}): ")

    if password == correct_password:
        print("Access granted!")
        break
    else:
        print("Incorrect password.")

if attempts == max_attempts and password != correct_password:
    print("Account locked. Too many failed attempts.")
```

Lab — Password Prompt

Time: 25 minutes

Task: Build a password prompt with retry logic.

Rules:

- Correct password: `'secure123'` (hardcoded for this lab)
- Maximum 3 attempts
- After each incorrect attempt, show how many attempts remain
- On success: print `'Access granted!'` and stop immediately
- After 3 failures: print `'Account locked. Contact support.'`

Optional: If the user enters an empty string, don't count it as an attempt — re-prompt instead.

Completion Criteria:

Common Pitfalls — Hour 26

⚠️ **Infinite loop** — forgetting to update the loop variable; `attempts` never changes, condition stays True forever

⚠️ **No `break` on success** — program prints "Access granted!" but keeps looping; must `break` immediately

⚠️ **Wrong condition** — `while attempts <= 3` gives 4 attempts instead of 3; use `<` not `<=`

⚠️ **Incrementing in the wrong place** — if you only increment on failure, the counter is wrong when the user succeeds early

Quick Check — Hour 26

Question: What is the minimum number of times a while loop's body can execute? What is the maximum?

Hint: What happens if the condition is False the very first time Python checks it?

Hour 27: for Loops + range()

Learning Outcomes

- Write a for loop that iterates over a list or other sequence
- Use `range(n)`, `range(start, stop)`, and `range(start, stop, step)` to generate sequences
- Explain why range() uses exclusive endpoints and how to adjust for inclusive ranges
- Choose between for loops and while loops based on whether you know the iteration count
- Build a multiplication table using nested for loops with range()

for Loops — Iterating Over Sequences

Structure

```
for item in sequence:
    # code to repeat once per item
```

Simple Example

```
names = ['Alice', 'Bob', 'Charlie']

for name in names:
    print(f'Hello, {name}!')
```

Output: `Hello, Alice!`, `Hello, Bob!`, `Hello, Charlie!`

Python moves through the sequence automatically — no counter to manage, no index to increment. **Prefer for over while when you know what you are iterating over.**

range() — Three Forms

range(stop) — start at 0, stop before n

```
for i in range(5):
    print(i)
# Output: 0, 1, 2, 3, 4
```

range(start, stop) — start at start, stop before stop

```
for i in range(2, 5):
    print(i)
# Output: 2, 3, 4
```

range(start, stop, step) — custom increment or countdown

```
for i in range(0, 10, 2):
    print(i)
# Output: 0, 2, 4, 6, 8
```

range() — Exclusive Endpoints

The Rule

`range(start, stop)` goes up to but **does NOT include** `stop`.

```
# Want 1 through 10? range(1, 10) gives only 1-9
for i in range(1, 10):
    print(i)   # Missing 10!

# Fix: use range(1, 11)
for i in range(1, 11):
    print(i)   # 1 through 10 inclusive
```

Why Exclusive?

- Length is obvious: `range(5)` gives exactly 5 numbers (0–4)
- Ranges fit together cleanly: `range(0, 5)` and `range(5, 10)` have no gap or overlap

Nested for Loops

What Are They?

A loop inside another loop. The inner loop completes **all** its iterations for each single iteration of the outer loop.

```
for row in range(3):
    for col in range(4):
        print(f'({row}, {col})', end=' ')
    print()  # new line after each row
```

Output:

```
(0, 0) (0, 1) (0, 2) (0, 3)
(1, 0) (1, 1) (1, 2) (1, 3)
(2, 0) (2, 1) (2, 2) (2, 3)
```

Total iterations: outer × inner = 3 × 4 = 12. Keep nesting to two levels when possible.

Demo — Multiplication Table

```python
# Multiplication table: 1 through 10
# Source: Day7_Hour3_Basics.md, section 7.2

print("Multiplication Table (1-10)")
print()

for i in range(1, 11):
    for j in range(1, 11):
        product = i * j
        print(f"{product:4}", end="")
    print()  # new line after each row
```

- `range(1, 11)` gives 1 through 10 (11 is exclusive)
- `f"{product:4}"` pads to width 4 so columns align
- `end=""` keeps products on one line; `print()` after inner loop starts the next row

Lab — Multiplication Table

Time: 25 minutes

Task: Ask the user for a number `n`, then print a multiplication table from 1 to n.

Rules:

- Prompt user for n (a positive integer)
- Print a table showing `i × j` for all i and j from 1 to n
- Format output so columns align (use `{:4}` or similar)

Optional: Add row and column headers (the numbers 1 through n).

Completion Criteria:

- Table prints correctly for n=5, n=10, n=12
- Columns align with a format specifier

Common Pitfalls — Hour 27

⚠️ **Wrong range** — `range(n)` gives 0 to n-1, not 1 to n; use `range(1, n + 1)` for 1 through n inclusive

⚠️ **No newline after row** — forgetting `print()` after the inner loop prints everything on one long line

⚠️ **Misaligned columns** — not using a format specifier like `{:4}` makes the table unreadable

⚠️ **Confusing i and j** — outer loop variable controls rows; inner loop variable controls columns

Quick Check — Hour 27

Question: What does `range(1, 4)` produce? What about `range(4, 1)`? What about `range(4, 1, -1)`?

Hour 28: Loop Patterns — Counters, Accumulators, Min/Max

Learning Outcomes

- Implement counter patterns — increment inside a conditional to count specific items
- Build accumulators — initialize and update variables that collect values across iterations
- Develop min/max tracking — find minimum and maximum values while iterating
- Calculate averages — combine counter and accumulator to compute the mean
- Build a complete number statistics program integrating all four patterns

Counter Pattern

What It Does

Counts how many items satisfy a condition. Initialize to 0; increment **inside** the `if` block.

```
numbers = [10, 5, 20, 15, 30]
even_count = 0

for num in numbers:
    if num % 2 == 0:    # check if even
        even_count = even_count + 1

print(f'Even numbers: {even_count}')  # Output: 3
```

Common mistake: putting `count = count + 1` outside the `if` — it then counts every iteration, not just matching items.

Accumulator Pattern

What It Does

Collects values across a loop. Choose the right initial value for the operation.

```
numbers = [10, 20, 30, 40, 50]
total = 0

for num in numbers:
    total = total + num

print(f'Sum: {total}')  # Output: 150
```

Initialization Rules

- **Addition** → initialize to `0`
- **Multiplication** → initialize to `1`
- **String concatenation** → initialize to `""`

Min/Max Pattern

What It Does

Tracks extreme values. Initialize with the **first item** — never with 0.

```
numbers = [10, 5, 20, 15, 30]
minimum = numbers[0]   # start with a real value from the data
maximum = numbers[0]

for num in numbers[1:]:   # skip first item (already assigned)
    if num < minimum:
        minimum = num
    if num > maximum:
        maximum = num

print(f'Min: {minimum}, Max: {maximum}')  # Min: 5, Max: 30
```

If you initialize `minimum = 0` and all values are positive, minimum stays 0 — a value that was never in the list.

Average Pattern

What It Does

Combines counter + accumulator. Divide **after** the loop completes — never inside it.

```
numbers = [10, 5, 20, 15, 30]
total = 0
count = 0

for num in numbers:
    total = total + num
    count = count + 1

average = total / count
print(f'Average: {average:.2f}')  # Output: 16.00
```

Format to two decimal places for clarity. Watch for division by zero if the list could be empty.

Demo — Number Statistics Program

```python
# Number Statistics Program
# Source: Day7_Hour4_Basics.md, section 4

numbers = [10, 5, 20, 15, 30]

total = 0
count = 0
minimum = numbers[0]
maximum = numbers[0]

for num in numbers[1:]:
    total = total + num
    count = count + 1
    if num < minimum:
        minimum = num
    if num > maximum:
        maximum = num

# Add back the first number (used for min/max init but not yet in total)
total = total + numbers[0]
count = count + 1

average = total / count

print(f'Min: {minimum:.1f}')
print(f'Max: {maximum:.1f}')
print(f'Sum: {total:.1f}')
print(f'Average: {average:.2f}')
```

Lab — Number Statistics

Time: 25 minutes

Starter data: `numbers = [12, 8, 25, 9, 18, 14, 22]`

Task: Complete a statistics program for this list.

Steps:

1. Initialize `total = 0`, `count = 0`, `minimum = numbers[0]`, `maximum = numbers[0]`
2. Write a for loop — update total, count, minimum, and maximum for every number
3. Calculate `average = total / count` after the loop
4. Print: Minimum, Maximum, Sum, Average

Completion Criteria:

Common Pitfalls — Hour 28

⚠️ **Counter increments every iteration** — increment is outside the `if` block; move it inside so it only counts matching items

⚠️ **Min/Max initialized to 0** — if all values are positive, `minimum` stays 0, which was never in the list; use `numbers[0]`

⚠️ **Average calculated inside the loop** — total and count are incomplete mid-loop; compute average only after the loop ends

⚠️ **First number missing from total** — if you start the loop at `numbers[1:]`, remember to add `numbers[0]` to total and count afterward

Quick Check — Hour 28

Question 1: In a counter pattern, does the increment go inside or outside the `if` block?

Question 2: You have a list of test scores. To find the lowest score, do you initialize `minimum` to 0 or to the first score?

Question 3: Why do we compute the average after the loop, not inside it?

Session 7 Wrap-Up

What We Covered Today

Hour 25 — Conditionals

- if/elif/else; first match wins rule
- Condition ordering: most specific to least specific
- Boundary values: `<=` vs `<`, testing at exact boundary inputs

Hour 26 — while Loops

- Repeats as long as condition is True; update variable to avoid infinite loops
- `break` for early exit; `continue` to skip an iteration
- Sentinel pattern: `while True` + `break`

Hour 27 — for Loops

Scope Guardrail

What We Are Building Toward

- Conditional logic with clear, ordered branches
- while loops with sentinel patterns
- for loops with range() and sequence iteration
- Core loop patterns: counter, accumulator, min/max, average

Not Yet Covered — Basics Course

- Defining and calling functions (Session 8)
- Importing modules (random, math, statistics)

Not Yet Covered — Advanced Course

- List comprehensions or generator expressions

Next Session Preview

Session 8 (Hours 29–32)

- Hour 29: Functions — defining, calling, returning values
- Hour 30: Function parameters and scope
- Hour 31: Functions with data structures
- Hour 32: Checkpoint review and practice

Preparation

- Think about how to break today's lab programs into smaller, named pieces
- Practice combining conditionals with loops in a single program
- Note: everything in Session 7 becomes a building block inside functions

Questions?

Remember:

- Order `elif` conditions from most restrictive to least restrictive
- Every while loop must change something that makes the condition eventually False
- `range(1, n + 1)` when you want 1 through n inclusive
- Initialize min/max to the first element, not to 0

Keep practicing — see you in Session 8!